

Звіт

до Лабораторної роботи №12

Stream API в Java

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Мета роботи

Метою лабораторної роботи є отримання практичних навичок використання Stream API у Java для обробки колекцій даних.

У роботі потрібно показати фільтрацію, перетворення, сортування, підрахунок, групування та збирання результатів у зручному вигляді.

2. Варіант роботи

Варіант 5 — медіатека

Для обраного варіанта було реалізовано консольну Java-програму для роботи з медіатекою. У програмі використовуються фільми, подкасти та епізоди подкастів.

3. Опис предметної області

У лабораторній роботі реалізовано просту модель медіатеки. Медіатека містить список фільмів і список подкастів. Кожен подкаст має власний список епізодів.

Над цими даними виконуються різні операції через Stream API: відбір фільмів за жанром, сортування за рейтингом, підрахунок, групування, пошук найдовшого фільму та побудова текстового звіту.

У проєкті реалізовано такі класи:

- Movie — фільм;
- Episode — епізод подкасту;
- Podcast — подкаст;
- MediaLibrary — сервіс для обробки даних через Stream API;
- Main — демонстраційний клас.

4. Класи предметної області

4.1 Клас Movie

Клас Movie описує фільм.

Поля класу:

- title — назва фільму;
- genre — жанр;
- year — рік випуску;
- rating — рейтинг;
- durationMinutes — тривалість у хвилинах.

Ці поля використовуються у stream-операціях для фільтрації, сортування, підрахунку та агрегування.

4.2 Клас Episode

Клас Episode описує епізод подкасту.

Поля класу:

- title — назва епізоду;
- seasonNumber — номер сезону;
- episodeNumber — номер епізоду;
- durationMinutes — тривалість епізоду.

4.3 Клас Podcast

Клас Podcast описує подкаст.

Поля класу:

- title — назва подкасту;
- host — ведучий;
- topic — тема;
- episodes — список епізодів.

Оскільки подкаст містить список епізодів, у роботі можна показати використання flatMap() для отримання всіх епізодів з усіх подкастів.

5. Клас MediaLibrary

Клас MediaLibrary є основним сервісом лабораторної роботи.

Він зберігає:

```
private final List<Movie> movies;
```

```
private final List<Podcast> podcasts;
```

У цьому класі реалізовано методи, які обробляють колекції через Stream API.

Основні методи:

- getMoviesByGenre(String genre);
- getMovieTitlesSortedByRating();
- getTopMovies(int limit);
- countMoviesByGenre(String genre);
- groupMoviesByGenre();
- countMoviesByGenreMap();
- getAverageMovieRating();
- getLongestMovie();
- getAllEpisodeTitles();
- buildMovieReport();
- getMoviesByGenreImperative(String genre).

6. Використані Stream API операції

Операція	Призначення у роботі
filter()	Відбір фільмів за жанром.

map()	Перетворення об'єктів у назви.
sorted()	Сортування фільмів за рейтингом.
count()	Підрахунок кількості фільмів певного жанру.
toList()	Збирання результату у список.
collect()	Збирання результату через колектори.
groupingBy()	Групування фільмів за жанром.
flatMap()	Отримання всіх епізодів з усіх подкастів.
max()	Пошук найдовшого фільму.
joining()	Побудова текстового звіту.

7. Приклади stream-методів

Метод фільтрації фільмів за жанром:

```
public List<Movie> getMoviesByGenre(String genre) {
    return movies.stream()
        .filter(movie -> movie.getGenre().equalsIgnoreCase(genre))
        .toList();
}
```

Метод отримання назв фільмів, відсортованих за рейтингом:

```

public List<String> getMovieTitlesSortedByRating() {
    return movies.stream()
        .sorted(Comparator.comparingDouble(Movie::getRating).reversed())
        .map(Movie::getTitle)
        .toList();
}

```

Метод групування фільмів за жанром:

```

public Map<String, List<Movie>> groupMoviesByGenre() {
    return movies.stream()
        .collect(Collectors.groupingBy(Movie::getGenre));
}

```

Метод отримання назв усіх епізодів:

```

public List<String> getAllEpisodeTitles() {
    return podcasts.stream()
        .flatMap(podcast -> podcast.getEpisodes().stream())
        .map(Episode::getTitle)
        .toList();
}

```

8. Складніший конвеєр

У роботі реалізовано побудову текстового звіту за фільмами.

```

public String buildMovieReport() {
    return movies.stream()
        .sorted(Comparator.comparingDouble(Movie::getRating).reversed())
        .map(movie -> movie.getTitle() + " — " + movie.getRating())
        .collect(Collectors.joining("\n"));
}

```

У цьому методі використано сортування, перетворення об'єктів у рядки та об'єднання результату в один текст.

9. UML-діаграма

Для моделі було створено UML-діаграму. На ній показано класи Movie, Episode, Podcast, сервіс MediaLibrary і демонстраційний клас Main.

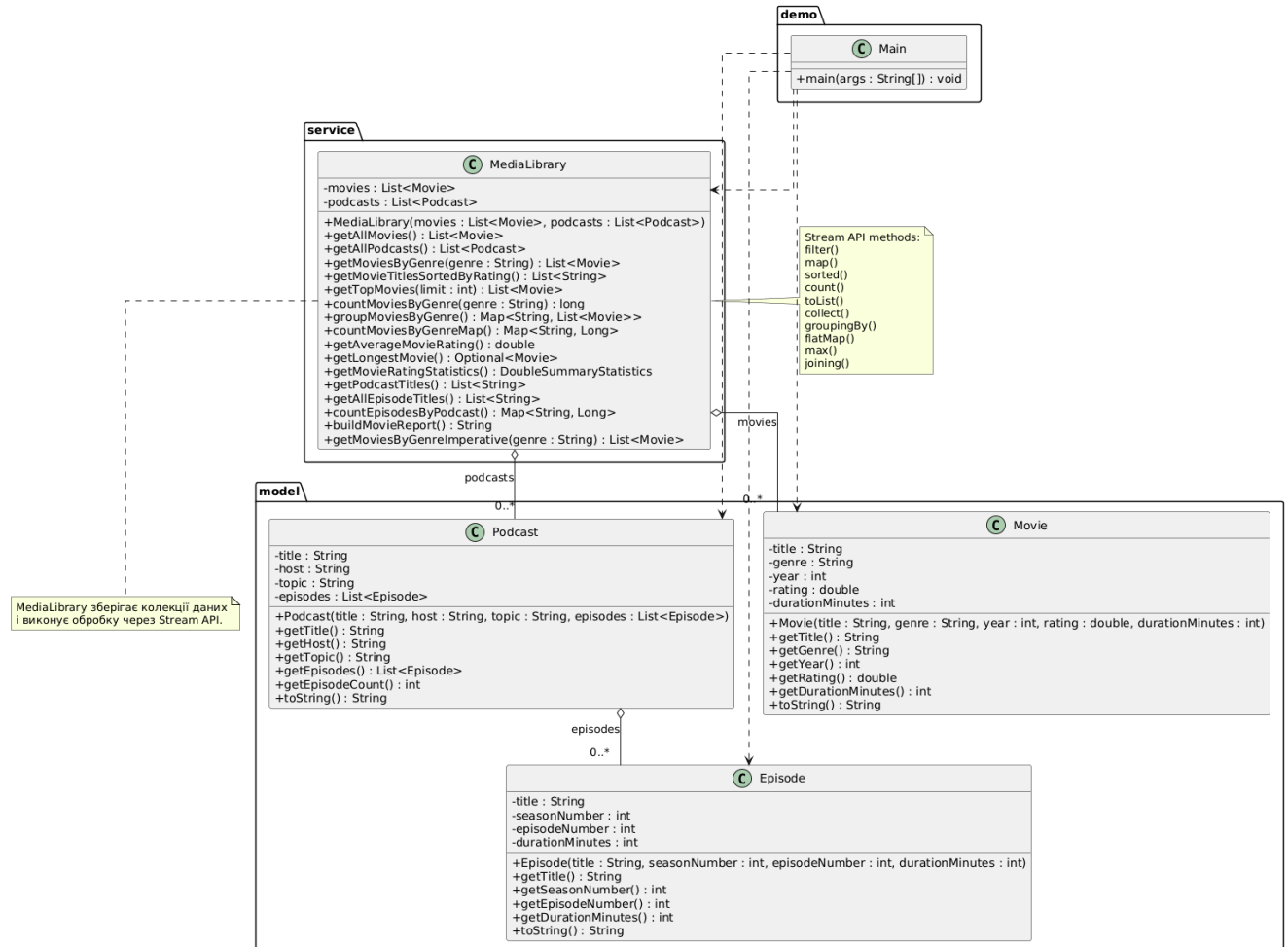


Рисунок 1 — UML-діаграма моделі медіатеки

10. Порівняння циклу і Stream API

Для порівняння було реалізовано метод пошуку фільмів за жанром через звичайний цикл:

```
public List<Movie> getMoviesByGenreImperative(String genre) {
    List<Movie> result = new ArrayList<>();

    for (Movie movie : movies) {
        if (movie.getGenre().equalsIgnoreCase(genre)) {
```

```

        result.add(movie);
    }
}

return result;
}

```

Аналогічний метод через Stream API:

```

public List<Movie> getMoviesByGenre(String genre) {
    return movies.stream()
        .filter(movie -> movie.getGenre().equalsIgnoreCase(genre))
        .toList();
}

```

Stream API краще показує намір програми: потрібно не описувати цикл вручну, а вказати, які саме елементи треба відібрати.

11. Демонстрація роботи

У класі Main створюються тестові дані: список фільмів, список епізодів і список подкастів.

Після цього створюється об'єкт MediaLibrary, через який демонструються основні stream-методи.

У демонстрації показано:

- фільтрацію фільмів за жанром;
- сортування назв фільмів за рейтингом;
- підрахунок фільмів;
- групування за жанром;
- обчислення середнього рейтингу;
- пошук найдовшого фільму;
- отримання назв епізодів;
- побудову текстового звіту;
- порівняння циклу і Stream API.

```
Stream API Media Library Demo
Усього фільмів: 5

Фільми жанру Фантастика
Фільм: Інтерстеллар, жанр: Фантастика, рік: 2014, рейтинг: 8.7, тривалість: 169 хв
Фільм: Початок, жанр: Фантастика, рік: 2010, рейтинг: 8.8, тривалість: 148 хв
Фільм: Матриця, жанр: Фантастика, рік: 1999, рейтинг: 8.7, тривалість: 136 хв

Назви фільмів за рейтингом
- Темний лицар
- Початок
- Форрест Гамп
- Інтерстеллар
- Матриця

Кількість фільмів жанру Драма
1

Групування фільмів за жанром
Бойовик: 1
Фантастика: 3
Драма: 1

Середній рейтинг фільмів
8.8

Найдовший фільм
Фільм: Інтерстеллар, жанр: Фантастика, рік: 2014, рейтинг: 8.7, тривалість: 169 хв
```

```
Усі епізоди подкастів
- Що таке Stream API
- Колекції в Java
- Фантастика в кіно
- Найкращі фільми 2000-х

Текстовий звіт за фільмами
Темний лицар — 9.0
Початок — 8.8
Форрест Гамп — 8.8
Інтерстеллар — 8.7
Матриця — 8.7

Порівняння циклу та Stream API
Через цикл: 3
Через Stream API: 3

Process finished with exit code 0
```

Рисунок 2 і 3 — результат запуску програми

12. Тестування

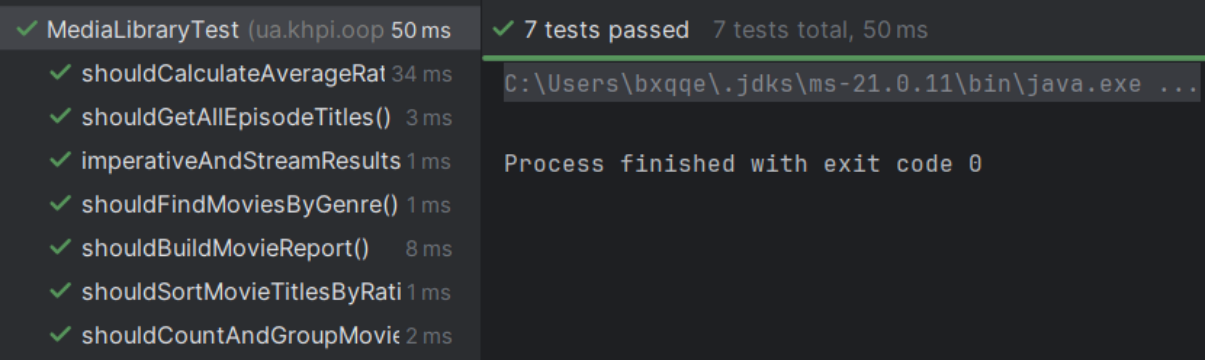
Для перевірки роботи програми були створені тести JUnit Jupiter.

Тестові класи:

- MediaLibraryTest;
- MainTest.

Тести перевіряють:

- пошук фільмів за жанром;
- сортування назв фільмів за рейтингом;
- підрахунок фільмів за жанром;
- групування фільмів за жанром;
- обчислення середнього рейтингу;
- пошук найдовшого фільму;
- отримання назв усіх епізодів;
- побудову текстового звіту;
- однаковість результатів імперативного підходу і Stream API;
- запуск Main без помилок.



The screenshot shows the results of running tests for the MediaLibraryTest class. On the left, a list of tests is shown with green checkmarks indicating they all passed. On the right, a summary shows that 7 tests passed out of 7 total, taking 50 ms. Below this, the command prompt shows the Java command used to run the tests, and the message 'Process finished with exit code 0' indicates successful completion.

Test Name	Duration
MediaLibraryTest	50 ms
shouldCalculateAverageRating	34 ms
shouldGetAllEpisodeTitles()	3 ms
imperativeAndStreamResults	1 ms
shouldFindMoviesByGenre()	1 ms
shouldBuildMovieReport()	8 ms
shouldSortMovieTitlesByRating	1 ms
shouldCountAndGroupMovies	2 ms

Summary: 7 tests passed, 7 tests total, 50 ms

Command: C:\Users\bxqqe\.jdk\ms-21.0.11\bin\java.exe ...

Process finished with exit code 0

Рисунок 4 — результат виконання тестів

13. Висновок

У ході виконання лабораторної роботи було реалізовано просту медіатеку з використанням Stream API.

Було створено класи Movie, Episode, Podcast та сервіс MediaLibrary.

У роботі використано основні можливості Stream API: filter(), map(), sorted(), count(), toList(), collect(), groupingBy(), flatMap(), max() і joining().

Stream API дозволив зробити обробку колекцій коротшою та зрозумілішою. Замість ручного керування циклами програма описує саму дію: відібрати, перетворити, відсортувати, згрупувати або підрахувати дані.